

# Conceitos de Criptografia

senhasegura

Departamento de Segurança e Inovação - SEGi9



**PÚBLICO**



## Resumo Executivo

Este texto tem como objetivo descrever os principais conceitos da área de segurança, com foco especial na área de criptografia.





## Sumário

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>Princípios</b>                            | <b>1</b>  |
| <b>2</b> | <b>Conhecimento Zero</b>                     | <b>2</b>  |
| 2.1      | Funcionamento do Conhecimento Zero . . . . . | 2         |
| <b>3</b> | <b>Confidencialidade</b>                     | <b>3</b>  |
| 3.1      | Algoritmo Simétrico . . . . .                | 3         |
| 3.2      | Algoritmo Assimétrico . . . . .              | 4         |
| 3.3      | Key-Encryption-Key (KEK) . . . . .           | 6         |
| 3.4      | Key Derivation Functions (KDF) . . . . .     | 9         |
| <b>4</b> | <b>Integridade</b>                           | <b>11</b> |
| 4.1      | Hash . . . . .                               | 11        |
| <b>5</b> | <b>Autenticidade</b>                         | <b>13</b> |
| 5.1      | Assinatura Digital . . . . .                 | 13        |
| <b>6</b> | <b>Confidencialidade e Autenticidade</b>     | <b>16</b> |





## 1 Princípios

A segurança da informação é um campo fundamental no mundo digital atual, onde a proteção dos dados e informações é essencial para a manutenção da privacidade e a confiabilidade das operações.

Dentro deste contexto, existem quatro pilares importantes que sustentam a segurança da informação: Conhecimento Zero, Confidencialidade, Integridade e Autenticidade. Esses pilares são fundamentais para garantir a proteção adequada dos dados e a confiança nas transações e comunicações. A seguir, vamos explorar cada um desses conceitos:

- Conhecimento Zero
- Confidencialidade
- Integridade
- Autenticidade





## 2 Conhecimento Zero

O conceito de Conhecimento Zero, ou Zero Knowledge, refere-se a um modelo de segurança onde o provedor de serviços não possui acesso às informações confidenciais armazenadas pelos usuários. Todas as credenciais, como senhas, notas seguras e outros dados sensíveis, são criptografadas localmente no dispositivo do usuário antes de serem enviadas e armazenadas nos servidores.

### 2.1 Funcionamento do Conhecimento Zero

#### 1. Criação e Armazenamento de Credenciais

- O usuário cria suas credenciais (senhas, notas seguras, etc.) no aplicativo.
- Antes de qualquer dado ser transmitido para os servidores, ele é criptografado localmente utilizando uma chave derivada da Senha Mestra do usuário.

#### 2. Criptografia Local

- A Senha Mestra é utilizada para gerar uma chave criptográfica.
- Esta chave criptográfica é então utilizada para criptografar todos os dados do usuário localmente no dispositivo.
- Os dados criptografados são enviados para os servidores para armazenamento seguro.

#### 3. Armazenamento Seguro

- Nos servidores, apenas os dados criptografados são armazenados.
- O servidor não possui acesso à Senha Mestra do usuário, nem à chave derivada da mesma. Isso significa que, mesmo que os servidores sejam comprometidos, os dados dos usuários permanecem inacessíveis e seguros.

#### 4. Descriptografia Local

- Quando o usuário precisa acessar suas credenciais, os dados criptografados são baixados do servidor e descriptografados localmente no dispositivo, utilizando novamente a Senha Mestra.
- Este processo garante que apenas o usuário, que possui conhecimento da Senha Mestra, pode acessar suas informações confidenciais.





### 3 Confidencialidade

A Confidencialidade refere-se à garantia de que as informações são acessíveis apenas por pessoas autorizadas. Em outras palavras, é a proteção contra a divulgação não autorizada de dados. Este pilar é essencial para manter a privacidade dos dados pessoais e sensíveis, como informações financeiras, médicas ou estratégicas de uma organização. Técnicas de criptografia são amplamente utilizadas para garantir a confidencialidade, assegurando que, mesmo que os dados sejam interceptados, eles não possam ser lidos ou compreendidos por indivíduos não autorizados.

#### 3.1 Algoritmo Simétrico

Entre os diversos tipos de algoritmos de criptografia, o algoritmo de criptografia simétrica é um dos mais comuns e amplamente utilizados. Neste tipo de algoritmo, a mesma chave( $K$ ) é utilizada tanto para criptografar( $C$ ), gerando a mensagem cifrada( $M$ ), quanto para descriptografar( $D$ ) os dados, gerando o Texto Plano( $T$ ).

A Figura 1 ilustra o funcionamento do algoritmo de criptografia simétrica. Para mostrar o funcionamento do algoritmo, vamos considerar três personagens: **Alice**, **Bob** e **Eva**. Alice e Bob desejam se comunicar de forma segura, enquanto Eva deseja ler a mensagem trocadas entre eles.

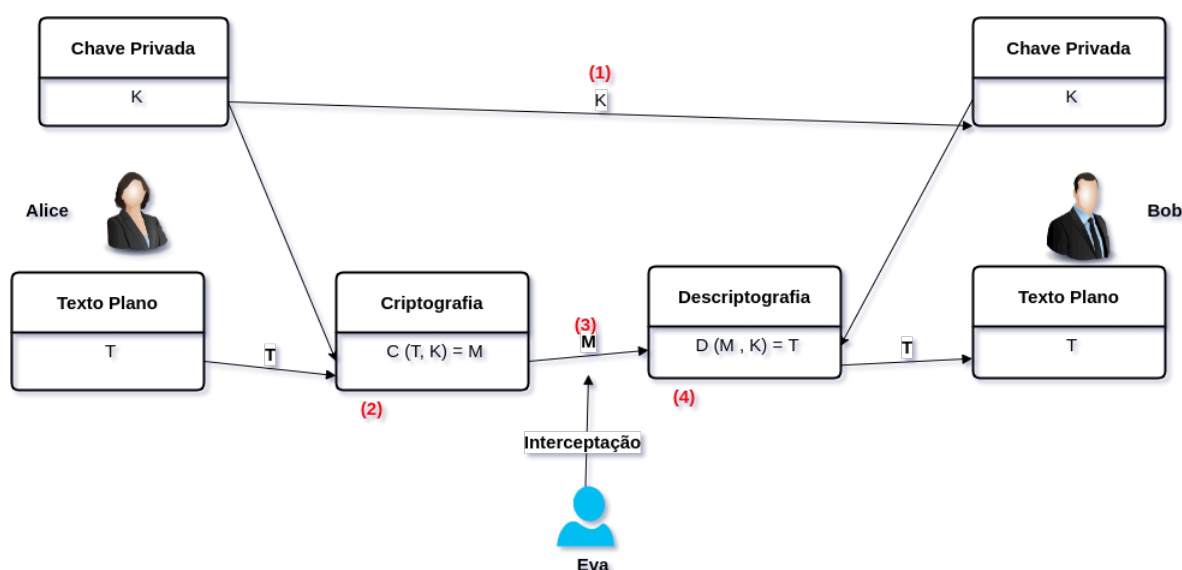


Figura 1: Funcionamento do algoritmo de criptografia simétrico





## Funcionamento do Algoritmo de Criptografia Simétrico

1. Alice e Bob primeiro estabelecem uma chave secreta(K) que ambos irão usar para criptografar e descriptografar as mensagens.

```
$ openssl rand -out K.key 32
```

2. Quando Alice deseja enviar o Texto Plano(T) confidencial para Bob, ela utiliza o algoritmo de Criptografia(C) e a Chave Secreta(K) para transformar o Texto Plano(T) no o Texto Cifrado(M).

```
$ openssl enc -aes-256-cbc -salt -in T.txt -out M.enc -pass file:./K.key -pbkdf2
```

3. Alice então envia Texto Cifrado(M) para Bob através de um canal de comunicação inseguro. Eva, que está tentando interceptar a comunicação, pode conseguir acessar o Texto Cifrado(M), mas sem a chave secreta(K), ela não consegue compreender a mensagem original.
4. Ao receber o Texto Cifrado(M), Bob utiliza o mesmo algoritmo de Descriptografia(D) e a Chave Secreta(K) para transformar o Texto Cifrado(M) de volta ao Texto Plano(T).

```
$ openssl enc -d -aes-256-cbc -in M.enc -out T.txt -pass file:./K.key -pbkdf2
```

A principal vantagem dos algoritmos de criptografia simétrica é sua eficiência e velocidade, especialmente em comparação com os algoritmos de criptografia assimétrica. No entanto, um dos desafios mais significativos é a distribuição segura da chave secreta. Se Eva conseguir interceptar ou descobrir a chave secreta, toda a comunicação entre Alice e Bob será comprometida.

Pelo motivo descrito acima, é essencial que Alice e Bob utilizem métodos seguros para compartilhar a chave secreta e garantam que ela permaneça protegida contra acessos não autorizados.

### 3.2 Algoritmo Assimétrico

O algoritmo de criptografia assimétrica é uma técnica fundamental no campo da segurança e criptografia. Diferente dos algoritmos de criptografia simétrica, que utilizam a mesma chave para criptografar e descriptografar os dados, os algoritmos assimétricos usam um par de chaves: uma chave pública(KPu) e uma chave privada(KPr). Vamos explorar o funcionamento desse algoritmo através dos personagens Alice, Bob e Eva exemplificado pela Figura 2.



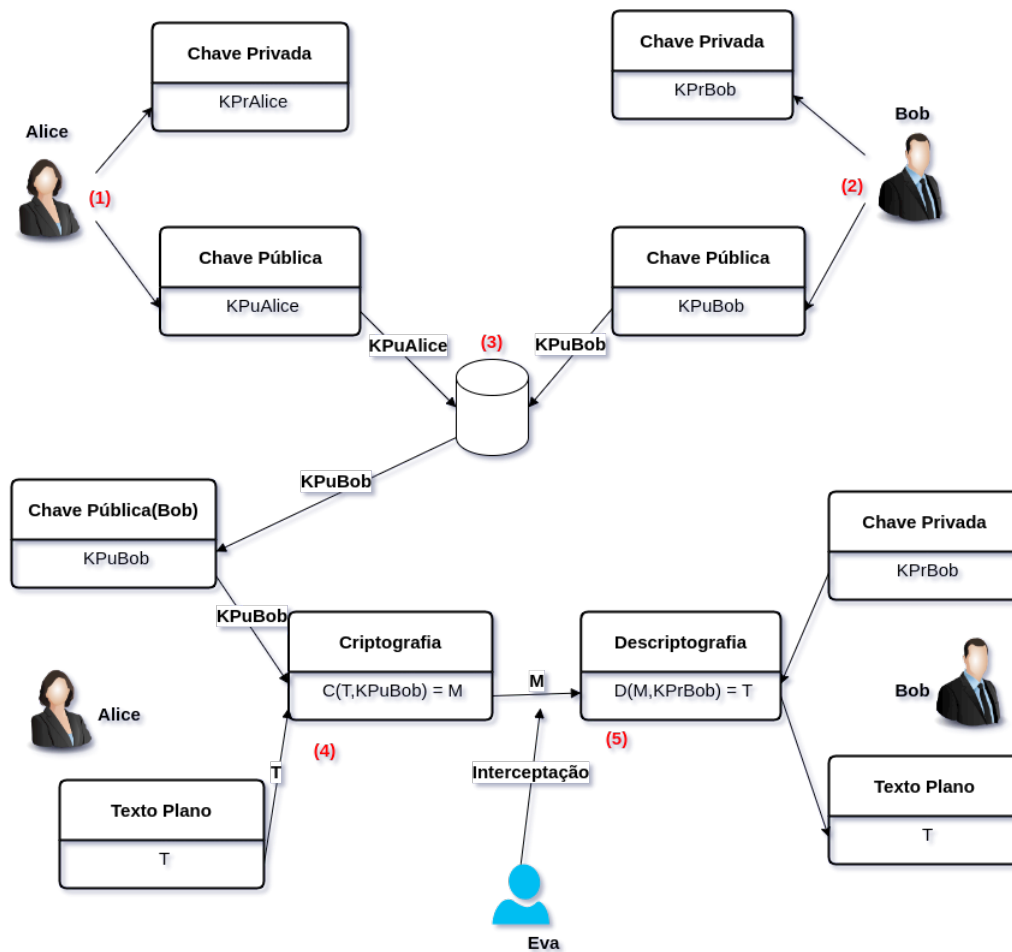


Figura 2: Funcionamento do algoritmo de criptografia assimétrico





## Funcionamento do Algoritmo de Criptografia Assimétrico

1. Alice gera um par de chaves, Chave Pública(KPuAlice) e Chave Privada(KPrAlice);

```
$ openssl genpkey -algorithm RSA -out KPrAlice.pem -pkeyopt rsa_keygen_bits:2048  
$ openssl rsa -pubout -in KPrAlice.pem -out KPuAlice.pem
```

2. Bob gera um par de chaves, Chave Pública(KPuBob) e Chave Privada(KPrBob);

```
$ openssl genpkey -algorithm RSA -out KPrBob.pem -pkeyopt rsa_keygen_bits:2048  
$ openssl rsa -pubout -in KPrBob.pem -out KPuBob.pem
```

3. Tanto Alice quanto Bob enviam as suas respectivas chaves públicas, KPuAlice e KPuBob, para um servidor de chave pública;
4. Quando Alice deseja enviar o Texto Plano(T) para Bob, ela utiliza a Chave Pública do Bob(KPuBob.pem) para gerando o Texto Cifrado(M);

```
$ openssl pkeyutl -encrypt -inkey KPuBob.pem -pubin -in T.txt -out M.bin
```

5. Ao receber o Texto Cifrado(M), o Bob utiliza sua Chave Privada(KPrBob) para descriptografá-la obtendo o Texto Plano(T).

```
$ openssl pkeyutl -decrypt -inkey KPrBob.pem -in M.bin -out T.txt
```

A segurança do algoritmo de criptografia assimétrica reside no fato de que, mesmo que Eva (uma personagem mal-intencionada) intercepte a mensagem criptografada, ela não conseguirá descriptografá-la sem a chave privada de Bob. Desta forma, a confidencialidade da comunicação entre Alice e Bob é mantida.

### 3.3 Key-Encryption-Key (KEK)

O conceito de Key-Encryption-Key(KEK), ou Chave de Criptografia de Chaves, é uma tecnologia utilizada para proteger outras chaves criptográficas, como por exemplo, o uso da KEK para proteger as Chaves de Criptografia de Dados(DEK). Ao usar uma KEK para resguardar as DEKs, adiciona-se uma camada extra de segurança ao sistema, melhorando significativamente sua proteção como um todo.

A Figura 3 ilustra o funcionamento e da utilização do conceito KEK.



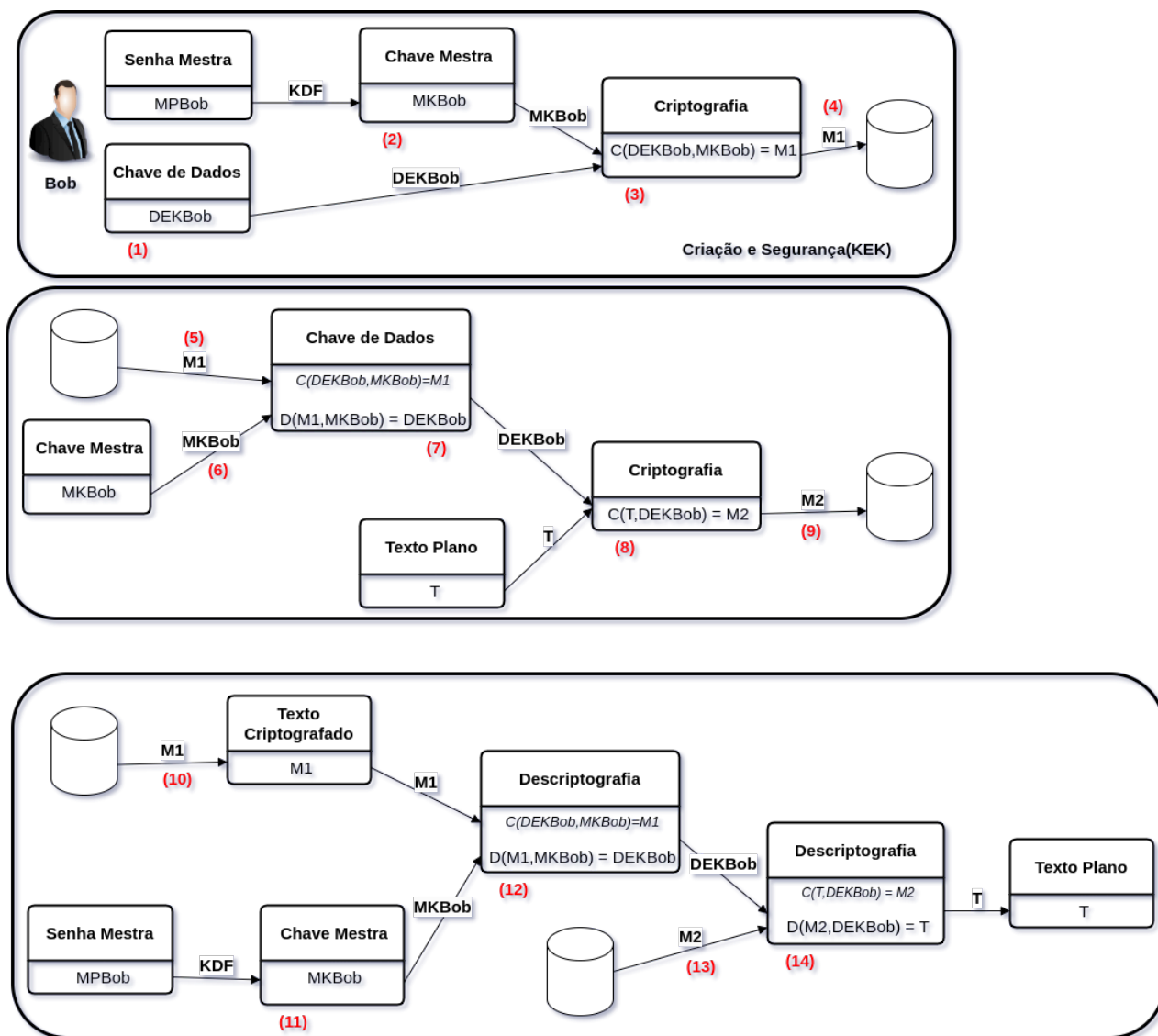


Figura 3: Funcionamento do conceito Key-Encryption-Key (KEK)

### - Criptografar a Chave de Dados(DEKBob)

1. Bob criar a Chave de Dados(DEKBob.pem);

```
$ openssl rand -out DEKBob.pem 32
```

2. Bob cria a Chave Mestra(MKBob.txt) baseada na Senha Mestra(MPBob);





```
# Define a senha mestra e a salt
$ MASTER_PASSWORD="MPBob"
$ SALT=$(openssl rand -hex 8) # Gera uma salt de 8 bytes (16 caracteres hexadecimais)

# Gere a chave e o IV, salvando a saída em um arquivo
$ openssl enc -aes-256-cbc -k "$MASTER_PASSWORD" -P -md sha256 -S "$SALT" -nosalt -iter 10000 > MKBob.txt
```

3. Aplicando o conceito de KEK, a Chave de Dados(DEKBob.pem) é criptografada com a Chave Mestra do Bob(MKBob.txt) gerando o Texto Cifrado M1;

```
$ openssl enc -aes-256-cbc -in DEKBob.pem -out M1.enc -pass pass:"$MASTER_PASSWORD" -S "$SALT" -md sha256 -iter 10000
```

4. Bob envia o Texto Cifrado M1 para o servidor;

#### - Criptografar o Texto Plano(T) com uma Chave de Dados DEKBob

5. Bob recupera o Texto Cifrado M1 do servidor;
6. Bob possui a sua Chave Mestra(MKBob.txt);
7. Bob descriptografa o Texto Cifrado M1 para ter o acesso a sua Chave de Dados(DEKBob.pem);

```
$ openssl enc -d -aes-256-cbc -in M1.enc -out DEKBob.pem -pass pass:"$MASTER_PASSWORD" -S "$SALT" -md sha256 -iter 10000
```

8. Bob utiliza a sua Chave de Dados(DEKBob.pem) para cifrar o Texto Plano(T) e gerando o Texto Cifrado M2;

```
$ openssl enc -aes-256-cbc -in T.txt -out M2.enc -pass file:DEKBob.pem -pbkdf2
```

9. Bob envia o Texto Cifrado M2 para o servidor;

#### - Descriptografar o Texto Cifrado M2 para obter o Texto Plano(T)

10. Bob recupera o Texto Cifrado M1 do servidor;
11. Bob possui a sua Chave Mestra(MKBob.txt);
12. Bob descriptografa o Texto Cifrado M1 com a sua Chave Mestra(MKBob.txt) para ter o acesso a sua Chave de Dados(DEKBob.pem);

```
$ openssl enc -d -aes-256-cbc -in M1.enc -out DEKBob.pem -pass pass:"$MASTER_PASSWORD" -S "$SALT" -md sha256 -iter 10000
```

13. Bob recupera o Texto Cifrado M2 do servidor;





14. Bob descriptografa o Texto Cifrado M2 com a sua Chave de Dados(DEKBob) para ter o acesso ao Texto Plano(T);

```
$ openssl enc -d -aes-256-cbc -in M2.enc -out T.txt -pass file:DEKBob.pem -pbkdf2
```

O Key-Encryption-Key (KEK) é um componente essencial em arquiteturas de segurança que exigem a proteção de múltiplas chaves criptográficas **DEK**. Seu uso permite a proteção eficiente das DEKs, garantindo a segurança dos dados criptografados e simplificando a gestão das chaves. Implementar KEKs corretamente é necessário para manter a integridade e a confidencialidade dos dados em qualquer sistema seguro.

### 3.4 Key Derivation Functions (KDF)

As funções de derivação de chaves (Key Derivation Functions - KDF) são um componente essencial na segurança e criptografia de dados. Estas funções são utilizadas para transformar um segredo inicial, como uma senha ou uma chave de criptografia, em uma ou mais chaves criptográficas que podem ser usadas para cifrar dados ou autenticar mensagens.

O processo de derivação de chave geralmente envolve a aplicação de uma função matemática que transforma a entrada original em uma chave na saída. A Figura 4 exemplifica o processo de derivação da Senha Mestra em uma Chave Mestra. Esta transformação é feita para garantir que a chave resultante seja segura e apropriada para uso em operações criptográficas. Os KDFs são projetados para serem resistentes a ataques, como ataques de força bruta, onde um atacante tenta todas as combinações possíveis até encontrar a correta.

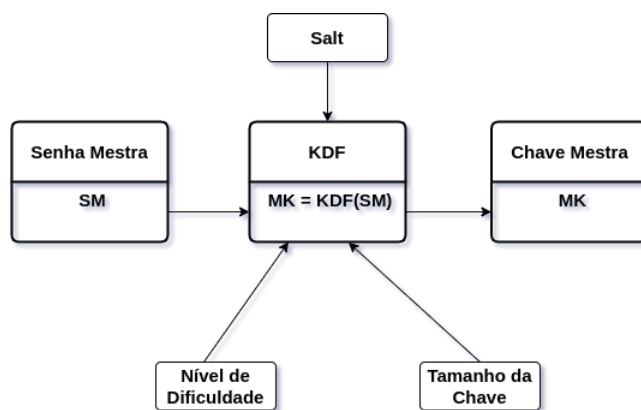


Figura 4: Funcionamento do conceito Key Derivation Functions (KDF)

Um exemplo comum de KDF é a função PBKDF2 (Password-Based Key Derivation Function 2), definida no padrão PKCS #5. O PBKDF2 usa um algoritmo de hash criptográfico, como





SHA-256, e um determinado número de iterações para transformar a senha inicial em uma chave derivada. O processo de iteração aumenta a dificuldade de ataques de força bruta, pois cada tentativa de senha requer várias operações de hash.

Além de PBKDF2, há também Argon2, uma KDF moderna que ganhou o Password Hashing Competition (PHC) em 2015. Argon2 é altamente configurável, permitindo ajustes precisos em termos de memória, tempo de processamento e paralelismo, oferecendo um equilíbrio entre segurança e performance.

O uso de KDFs é essencial em sistemas onde a segurança da chave criptográfica é crucial. Sem um KDF adequado, as chaves derivadas podem ser vulneráveis a diversos tipos de ataques, comprometendo a integridade e confidencialidade dos dados. Portanto, a escolha e implementação corretas de uma KDF robusta são fundamentais para a segurança criptográfica eficaz.

Abaixo estão exemplos de comandos que utilizam o OpenSSL para realizar o processo de Key Derivation Functions (KDF), que é usado para derivar uma chave criptográfica a partir de uma senha ou outra informação inicial.

1. Derivar uma chave usando PBKDF2 e usar essa chave para criptografar o arquivo texto-plano.txt, salvando a saída criptografada em derivedkey.bin

```
# Define a senha mestra e a salt
$ MASTER_PASSWORD="MPBob"
$ SALT=$(openssl rand -hex 8) # Gera uma salt de 8 bytes (16 caracteres hexadecimais)

# Gere a chave e o IV, salvando a saída em um arquivo
$ openssl enc -aes-256-cbc -k "$MASTER_PASSWORD" -P -md sha256 -S "$SALT" -nosalt -iter 10000 > MKBob.txt
```





## 4 Integridade

A Integridade diz respeito à precisão e consistência das informações ao longo de seu ciclo de vida. Este pilar assegura que os dados não sejam alterados de forma indevida, seja por erros acidentais ou por ações maliciosas. A integridade é mantida através de mecanismos de verificação, como somas de verificação, assinaturas digitais e funções de hash. Esses métodos permitem que qualquer alteração nos dados seja detectada, garantindo que a informação recebida é exatamente a mesma que foi enviada originalmente, sem modificações não autorizadas.

### 4.1 Hash

Um hash é uma função que transforma uma entrada de dados (que pode variar de alguns bytes a grandes volumes de dados) em uma saída de tamanho fixo, geralmente representada como uma sequência de caracteres alfanuméricos. Esta transformação é determinística, o que significa que a mesma entrada sempre produzirá a mesma saída.

Alguns exemplos de algoritmos hash populares incluem a família de algoritmos SHA-2 (Secure Hash Algorithm 2), que inclui SHA-224, SHA-256, SHA-384 e SHA-512. Os algoritmos SHA-2 produzem hashes de diferentes tamanhos, sendo o SHA-256 e o SHA-512 os mais comuns, gerando valores de 256 e 512 bits, respectivamente.

A Figura 5 exemplifica a utilização do algoritmo de hash para verificar a integridade de uma informação enviada de **Bob** para **Alice**.

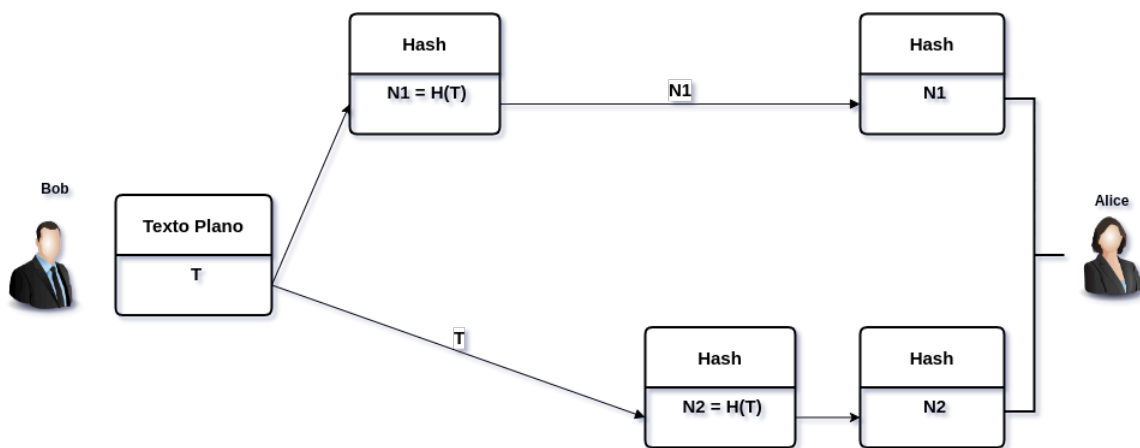


Figura 5: Funcionamento do processo de utilização do algoritmo Hash

Inicialmente, **Bob** executa o algoritmo de hash sobre a mensagem e obtém o que normalmente é chamado de resumo. Tanto a mensagem quanto o resumo são enviados para **Alice**. **Alice** gera novamente o algoritmo de hash sobre a mensagem e compara o resultado do hash com o





hash que foi enviado inicialmente por **Bob**. Se por acaso os valores forem iguais, então isso indica que não houve nenhum tipo de alteração na mensagem.

## Prático

Abaixo estão exemplos de comandos utilizando o OpenSSL para realizar o processo de hash, e como é utilizado para gerar uma representação fixa de dados, independente do tamanho e do conteúdo original.

1. Gerar um hash (hash.sha256) utilizando SHA-256

```
$ openssl dgst -sha256 -out hash.sha256 arquivo.txt
```

2. Verificar a integridade de um arquivo (arquivo.txt) comparando hashes

```
$ openssl dgst -sha256 arquivo.txt
```

3. Gerar um hash (hash.sha512) utilizando SHA-512

```
$ openssl dgst -sha512 -out hash.sha512 arquivo.txt
```

4. Verificar a integridade de um arquivo (arquivo.txt) comparando hashes

```
$ openssl dgst -sha512 arquivo.txt
```





## 5 Autenticidade

A Autenticidade está relacionada à verificação da identidade das partes envolvidas na comunicação e à garantia de que a origem das informações é confiável. Este pilar garante que as entidades (pessoas, sistemas ou dispositivos) sejam quem dizem ser e que as informações provenientes dessas entidades sejam genuínas. Métodos de autenticação, como senhas, certificados digitais e autenticação de dois fatores, são utilizados para validar a identidade e estabelecer a confiança entre as partes. A autenticidade é crucial para prevenir fraudes e assegurar a legitimidade das transações eletrônicas.

### 5.1 Assinatura Digital

A assinatura digital é um mecanismo de segurança criptográfica fundamental para garantir a integridade e a autenticidade das informações transmitidas digitalmente.

#### Conceito de Assinatura Digital

A assinatura digital utiliza algoritmos de **criptografia assimétrica** para garantir que uma mensagem ou documento não tenha sido alterado desde que foi assinado e que realmente foi enviado pelo remetente.

#### Funcionamento

A Figura 6 exemplifica o algoritmo de assinatura digital. Neste exemplo, Bob deseja enviar uma mensagem para Alice e Bob deseja garantir que Alice possa verificar a autenticidade da mensagem.



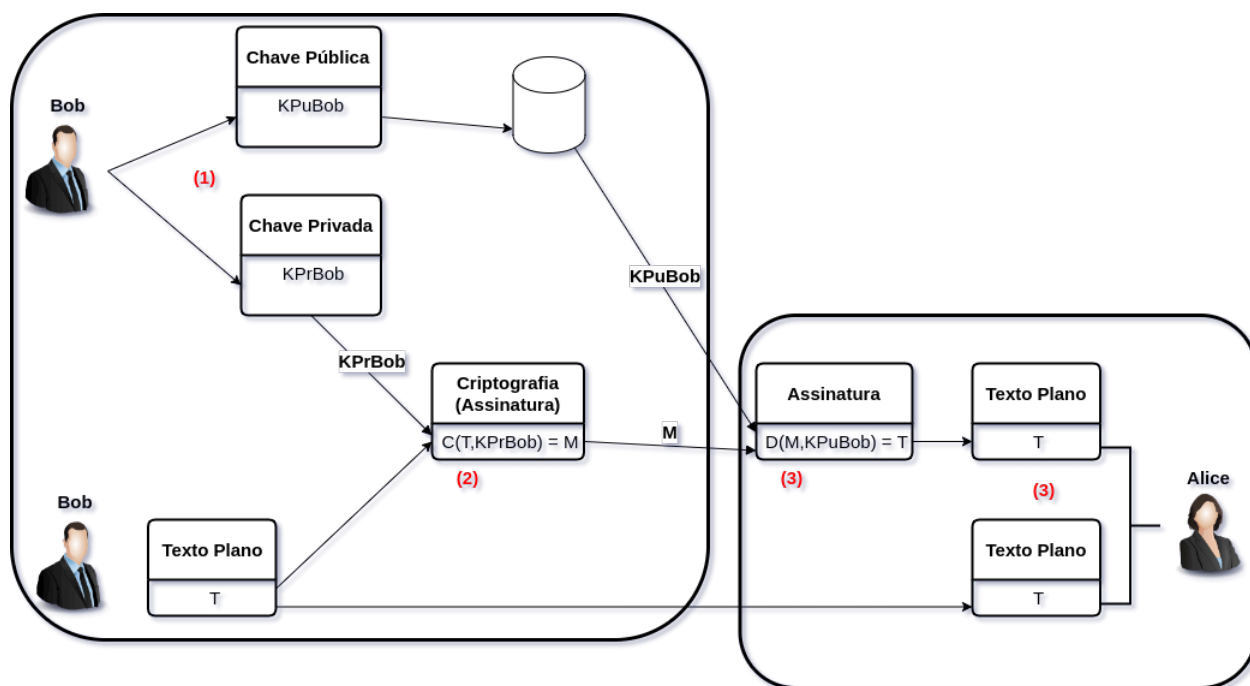


Figura 6: Funcionamento do processo de Assinatura Digital

1. **Criação de Chaves:** Bob, que deseja enviar uma mensagem assinada para Alice, primeiro cria um par de chaves criptográficas: uma chave pública(KPubBob) e uma chave privada(KPrBob);

```
$ openssl genpkey -algorithm RSA -out KPrBob.pem -pkeyopt rsa_keygen_bits:2048  
$ openssl rsa -pubout -in KPrBob.pem -out KPubBob.pem
```

2. **Assinatura da Mensagem:** Bob escreve sua mensagem e, em seguida, utiliza sua chave privada para gerar uma assinatura digital. Este processo envolve a aplicação de uma função de hash na mensagem para gerar um resumo da mesma (digest). Em seguida, Bob criptografa este resumo utilizando sua chave privada. O resultado é a assinatura digital, que é enviada juntamente com a mensagem original para Alice.

```
$ openssl dgst -sha256 -sign KPrBob.pem -out signature.bin arquivo.txt
```

3. **Verificação da Assinatura:** Ao receber a mensagem e a assinatura digital, Alice utiliza a chave pública de Bob para verificar a assinatura. Primeiro, ela aplica a mesma função de hash na mensagem recebida para gerar um resumo. Depois, ela utiliza a chave pública de Bob para descriptografar a assinatura digital recebida, obtendo o resumo original da mensagem assinado por Bob.





- Se o resumo gerado por Alice e o resumo obtido através da descryptografia da assinatura coincidirem, Alice pode ter certeza de que a mensagem não foi alterada e foi realmente assinada por Bob.
- Se os resumos não coincidirem, isso indica que a mensagem foi alterada no caminho ou que a assinatura não foi gerada pela chave privada de Bob.

```
$ openssl dgst -sha256 -verify KPuBob.pem -signature signature.bin arquivo.txt  
Verified OK
```

## Importância da Assinatura Digital

A assinatura digital proporciona dois aspectos cruciais de segurança:

- **Autenticidade:** Garante que a mensagem foi enviada pelo remetente legítimo (no caso, Bob).
- **Integridade:** Assegura que a mensagem não foi alterada durante a transmissão.

Em conjunto, esses aspectos permitem a confiança em comunicações eletrônicas, especialmente em transações financeiras, documentos legais e comunicação corporativa.

## Conclusão

A assinatura digital é um componente vital na segurança da informação, oferecendo uma maneira robusta de verificar a integridade e a autenticidade de dados digitais. No exemplo de Bob e Alice, vimos como a criptografia assimétrica é utilizada para gerar e verificar assinaturas digitais, garantindo a segurança nas comunicações digitais.





## 6 Confidencialidade e Autenticidade

Utilizando os processos de Confidencialidade e Autenticidade, Alice garante que a mensagem enviada pertence ao Bob e que a mensagem foi transportada de uma forma segura.

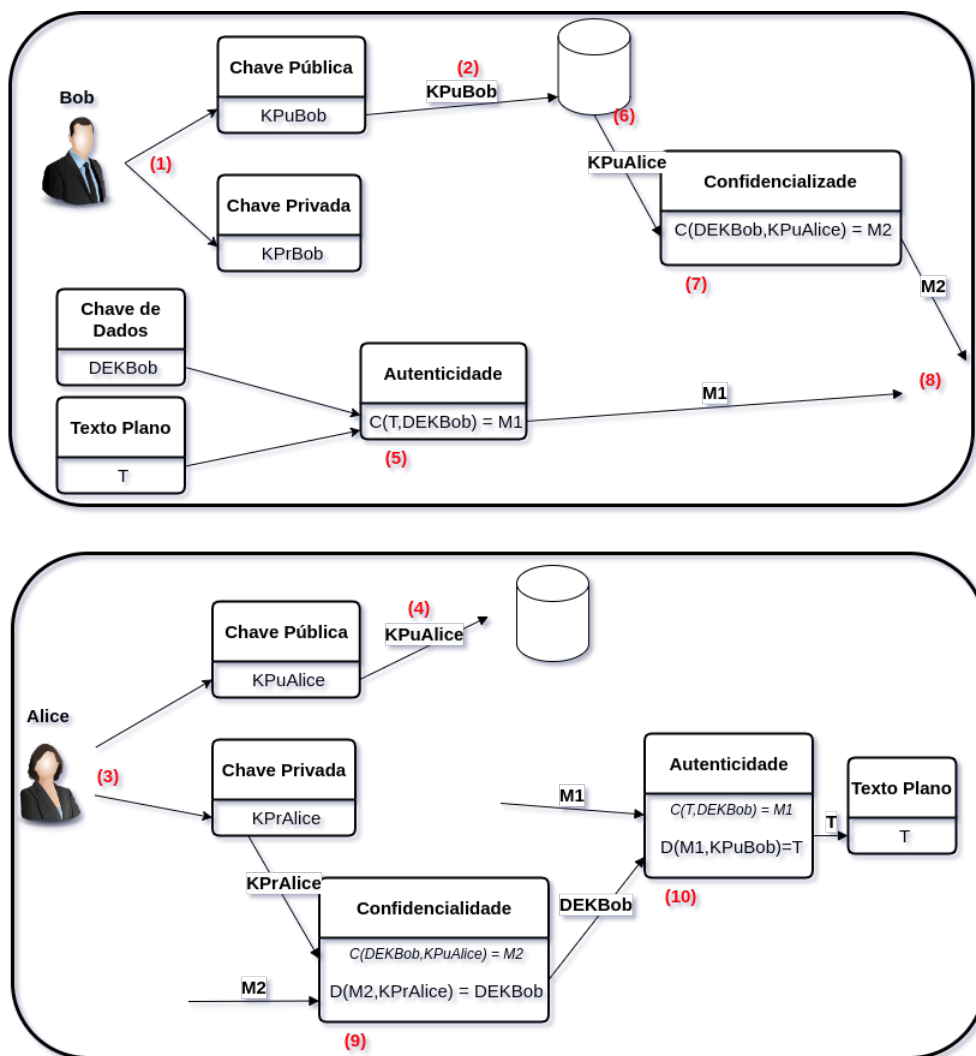


Figura 7: Confidencialidade e Autenticidade de uma mensagem

### - Criação das Chaves de Criptografia

1. Bob cria o seu par de chaves, Chave Pública(KPubBob.pem) e Chave Privada(KPrBob.pem);

```
$ openssl genpkey -algorithm RSA -out KPrBob.pem -pkeyopt rsa_keygen_bits:2048
$ openssl rsa -pubout -in KPrBob.pem -out KPubBob.pem
```





2. Bob envia a sua Chave Pública(KPUBob.pem) para o servidor;
3. Alice cria o seu par de chaves, Chave Pública(KPUAlice.pem) e Chave Privada(KPrAlice.pem);

```
$ openssl genpkey -algorithm RSA -out KPrAlice.pem -pkeyopt rsa_keygen_bits:2048
$ openssl rsa -pubout -in KPrAlice.pem -out KPUAlice.pem
```

4. Alice envia a sua Chave Pública(KPUAlice.pem) para o servidor;

#### - Assinatura(5) e Seguraça(7)

5. Bob aplica o conceito de KEK para gerar o Texto Cifrado M1;

```
# Gerar uma chave simétrica (DEKBob.bin)
$ openssl rand -out DEKBob.bin 32

# Criptografar T.txt usando DEKBob.bin
$ openssl enc -aes-256-cbc -salt -in T.txt -out M1.enc -pass file:./DEKBob.bin -pbkdf2
```

6. Bob recupera a Chave Pública da Alice(KPUAlice)
7. Bob cifra a sua Chave de Dados(DEKBob.bin) com a Chave Publica da Alice(KPUAlice.pem) e gerando o Texto Cifrado M2;

```
# Criptografar a chave AES usando a chave pública de Alice (KPUAlice.pem)
$ openssl pkeyutl -encrypt -in DEKBob.bin -inkey KPUAlice.pem -pubin -out M2.enc
```

8. Bob envia o Texto Cifrado M1 e M2 para o servidor;

#### - Seguraça(9) e Assinatura(10)

9. Alice descriptografa o Texto Cifrado M2 com a sua Chave Privada(KPrAlice) para obter a Chave de Dados do Bob(DEKBob.bin);

```
$ openssl pkeyutl -decrypt -in M2.enc -inkey KPrAlice.pem -out DEKBob.bin
```

10. Alice descriptografa o Texto Cifrado M1 com a DEK do Bob(DEKBob.bin) para obter o Texto Plano(T);

```
$ openssl enc -d -aes-256-cbc -in M1.enc -out T.txt -pass file:./DEKBob.bin -pbkdf2
```





Copyright

All rights reserved to MT4 Tecnologia LTDA.

Propósito deste documento

Conceitos de criptografia

Versão

Versão 1.0

Autor

Caio Abreu Ferreira [[cferreiras@senhasegura.com](mailto:cferreiras@senhasegura.com)]

MT4 Tecnologia Ltda.

Rua Joaquim Antunes, 767 – Pinheiros – São Paulo

Phone: +55 11 3065-7070

[www.mt4.com.br](http://www.mt4.com.br)

[support@senhasegura.com](mailto:support@senhasegura.com)

